Figure 2.4 The structure of a new Spring Boot project generated from Spring Initializr

In the code repository accompanying this book (https://github.com/ThomasVitale/cloud-native-spring-in-action), you can find "begin" and "end" folders for each chapter, so that you can always start with the same setup as me and check the final result. For example, you are currently reading chapter 2, so you will find the related code in Chapter02/02-begin and Chapter02/02-end.

TIP In the "begin" folder for this chapter, you'll find a `curl` command that you can run in a Terminal window to download a zip file containing all the code you need to get started, without going through the manual project generation on the Spring Initializr website.

Gradle or Maven?

I'm using Gradle in this book, but feel free to use Maven instead. In the code repository accompanying this book, you can find a table mapping the Gradle commands to Maven so that you can easily follow along, should you use the second option (https://github.com/ThomasVitale/cloud-native-spring-in-action). Each project has different needs that might lead you to choose one build tool over another.

My choice of using Gradle is a personal preference and is due to two main reasons. Building and testing a Java project with Gradle takes less time than Maven, thanks to its incremental and parallel builds and caching system. Also, I find the Gradle build language (Gradle DSL) to be more readable, expressive, and maintainable than Maven XML. In the Spring ecosystem, you can find projects using Gradle and others using Maven. They're both good choices. I recommend you try them both and pick the tool that makes you more productive.

EXPLORING THE BUILD CONFIGURATION

Open the project you have just initialized in your favorite IDE, and look at the Gradle build configuration for the Catalog Service application, defined in